

Project Proposal: Go-Karting Game

Project Description

The project is a **Go-Karting Game** designed to emulate go-karting experience of real life. Go-Kart is a small car used on controlled racing tracks that you can drive for fun. Players will navigate through multiple tracks of increasing difficulty, with progression determined by their performance, potentially related to factors the time taken to complete laps or a set number of laps. The game emphasizes physics-based realism, incorporating elements such as friction, gravity, and collisions, which will influence the car's movement and add an extra layer of challenge. Each track will feature a variety of obstacles, making navigation more complex and demanding strategic maneuvering. The game will have a top-down view, offering a clear perspective of the arena and the player's car.

Competitive Analysis

Several existing tutorials, such as space shooter games, adopt a top-view perspective and a dynamic camera that tracks the player's position in a large arena. Similarly, radar systems are often employed in these games to provide players with a view of the entire arena, tracking other entities such as enemies or allies. My game will build upon and differentiate itself from these approaches. While I will use a moving camera and a radar, arenas will be far more intricate, with predefined paths and diverse, curving tracks that introduce unique challenges, offering multiple levels and environments, each with distinct features and obstacles. The inclusion of physics elements, like friction and gravity, will further distinguish our game by adding a layer of realism often absent from similar games.

Structural Plan

The core *Car* class manages all car-related properties, such as position, velocity, angle, and movement, with methods for things like collision detection and drawing. Automated cars act as dynamic obstacles with predefined or randomized behaviors. The arena is represented as a large, manually designed space with polygonal boundaries to define tracks and ensure accurate collision handling. A *Camera* class dynamically tracks the player's car, ensuring it stays within a visible window, while a radar provides a scaled-down overview of the entire arena, displaying all cars and key features. To ensure seamless integration and expandability, the game includes an input handling system to manage player inputs, enabling smooth interactions and future support for multiplayer controls. A physics system governs mechanics like gravity, friction, and collisions, ensuring realistic movement. Game logic oversees lap completions, progression, and scoring while coordinating with an event system to handle critical in-game occurrences such as collisions or checkpoint crossings.

Algorithmic Plan

According to me, the trickiest part of the project is ensuring that the cars behave realistically while navigating curvy, complex tracks, thus making a singular code for handling all types of tracks and potential collisions. Although I am still thinking about the implementation issues, I am thinking of representing the tracks as a series of polygons and using the coordinates of relevant polygon edges (representing different sections of the track) for defining track boundaries. So, when a go-kart moves, an event function in the code will check its position, whether it's inside or outside the track. It will stop or bounce back the car if it crosses the track boundary.

Implementing go-kart's movement considering friction, acceleration and eliminating sudden direction changes is fairly challenging too, which can be done by adjusting velocity dynamically using trigonometric functions and certain fixed constants representing physics quantities. I am

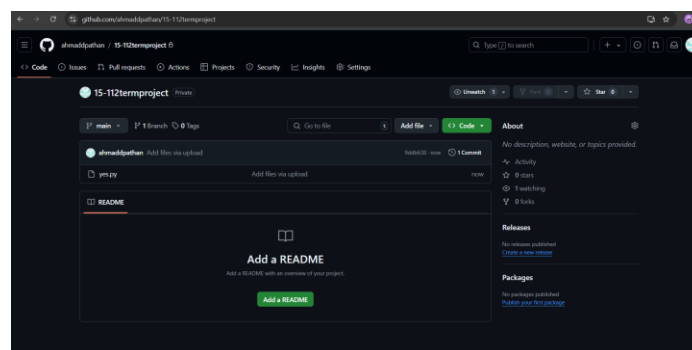
thinking of slowing the car down instead of bouncing it back upon collision for realistic experience.

Timeline Plan

By TP2, I will implement the moving camera and radar display, develop the core physics system, including gravity, friction, and collision detection, create multiple racing tracks with functional checkpoints and lap timers and design basic graphics and animations for the car and tracks. Post-TP2, I will introduce the multiplayer mode, allowing multiple players to race against each other, add more detailed arenas, obstacles, and environmental variations, potentially integrate Arduino for joystick support and refine graphics and animations for a more polished appearance.

Version Control Plan

I will use a private project repository on GitHub as the version control system. After every significant revision or modification to the codebase, I will commit the changes locally and push them to the GitHub repository. This approach will enable easy rollback to previous versions if any issues arise.



Module List

The game will exclusively use the cmu_graphics library for everything. However, once MVP is achieved, I plan to explore integrating Arduino to enable joystick control for a more immersive experience. I might even plan to employ the arcade box for the multiplayer mode.

TP2 Update

In the process of implementing the game, some changes and challenges arose, leading to significant updates. The only difficulty I am currently facing is implementing a proper collision system for the track and the car. Initially, I considered representing the tracks using groups of rectangles and arcs, but creating curves proved to be very challenging, so I decided not to proceed with this idea. Another approach involved defining equations for the outer and inner edges of the track and basing the collision system on these equations. However, I have not yet figured out how to find the equation for a complex, curvy, closed track, making this method difficult to implement. A simpler option is to use very basic shapes for tracks, which might simplify collision handling. I also experimented with implementing a collision system based on polygon tracks. Unfortunately, this approach was not successful as the results were both visually unappealing and prone to glitches in the collision system. These challenges reflect my ongoing efforts to ensure realistic behavior and an enjoyable gameplay experience.

TP3 Update

Since the last update, the concept of the project has undergone a significant transformation, evolving from a go-karting game to a goal-oriented driving game. Initially envisioned as a lap-based simulation, the project now revolves around collecting coins strategically scattered across the track. This shift not only streamlined the mechanics but also introduced more interactive and visually engaging gameplay elements.

The transition in gameplay mechanics represents the most substantial change. Rather than focusing on completing laps, the game challenges the player to collect coins within

a time limit to progress to the next level. A health system has replaced the previous penalty mechanics, where the player loses health when their vehicle strays off the track. If health depletes entirely before achieving the required score, the game ends.

The track design remains central to the gameplay, leveraging image processing techniques using PIL library to validate the placement of both coins and the vehicle. Coins are generated only within the track's boundaries using pixel-level RGB checks to ensure accuracy and gameplay integrity.

The visual representation of the car has been significantly enhanced. The project employs a 2D rotation formula to accurately calculate the car's corners, dynamically updating its appearance to reflect changes in position and orientation.

Another critical addition is the implementation of multiple screens to improve the overall gameplay flow. The scoring system also ensures meaningful player interaction, rewarding precise driving and strategic collection of coins. The game also features progressive difficulty with increasing coin targets, health penalties, and level complexity. This layered progression keeps the experience engaging as the player advances.